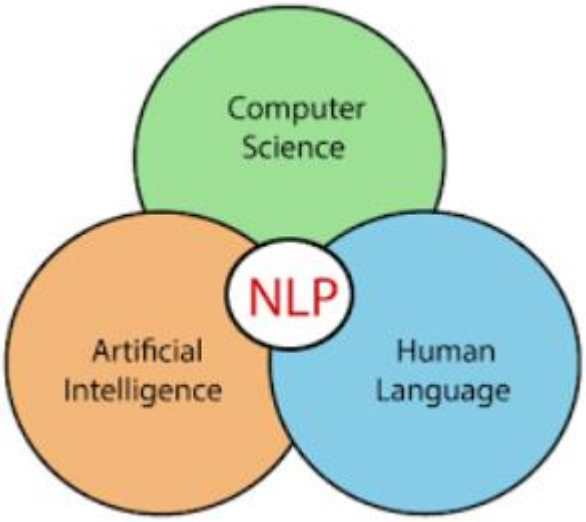
Introduction to Natural Language Processing

NLP stands for Natural Language Processing, which is a part of Computer Science, Human language, and Artificial Intelligence. It is the technology that is used by machines to understand, analyze, manipulate, and interpret human's languages. It builds systems that can process both spoken and written language, allowing for more intelligent human-machine interactions. It helps developers to organize knowledge for performing tasks such as translation, automatic summarization, Named Entity Recognition (NER), speech recognition, relationship extraction, and topic segmentation.



Advantages of NLP:

- NLP helps users to ask questions about any subject and get a direct response within seconds.
- NLP offers exact answers to the question means it does not offer unnecessary and unwanted information.
- NLP helps computers to communicate with humans in their languages.
- It is very time efficient.

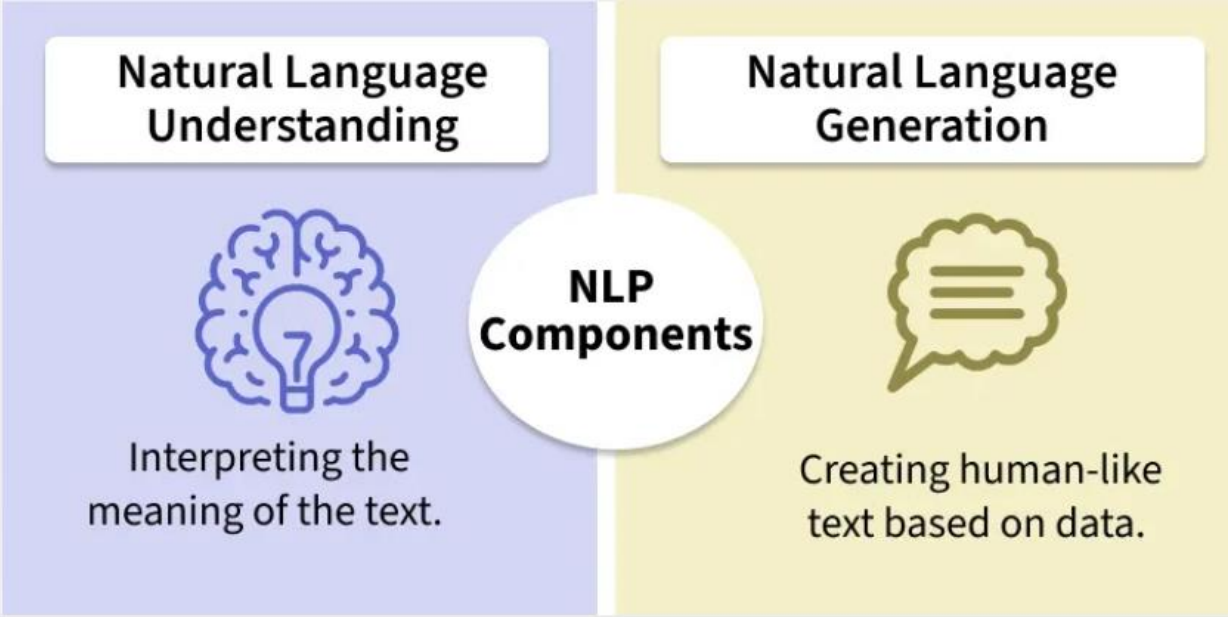
- Most of the companies use NLP to improve the efficiency of documentation processes, accuracy of documentation, and identify the information from large databases.

Disadvantages of NLP:

A list of disadvantages of NLP is given below:

- NLP may not show context.
- NLP is unpredictable.
- NLP may require more keystrokes.
- NLP is unable to adapt to the new domain, and it has a limited function that's why NLP is built for a single and specific task only.

Components of NLP:



There are the following two components of NLP -

1. Natural Language Understanding (NLU)

Natural Language Understanding (NLU) helps the machine to understand and analyze human language by extracting the metadata from content such as concepts, entities, keywords, emotion, relations, and semantic roles.

NLU mainly used in Business applications to understand the customer's problem in both spoken and written language.

NLU involves the following tasks -

- It is used to map the given input into useful representation.
- It is used to analyze different aspects of the language.

2. Natural Language Generation (NLG)

Natural Language Generation (NLG) acts as a translator that converts the computerized data into natural language representation. It mainly involves Text planning, Sentence planning, and Text Realization.

Text Planning:

This stage involves deciding what information needs to be included in the generated text often by retrieving relevant content from a knowledge base.

Sentence planning:

Here, the system selects appropriate words, form phrases and determines the sentence structure and tone.

Text Realization:

this is the stage we are focusing on, where the chosen words and structures are converted into a grammatically correct sentence.

Difference between NLU and NLG:

NLU	NLG
NLU is the process of reading and interpreting language.	NLG is the process of writing or generating language.
It produces non-linguistic outputs from natural language inputs.	It produces constructing natural language outputs from non-linguistic inputs.

Applications of NLP

There are the following applications of NLP -

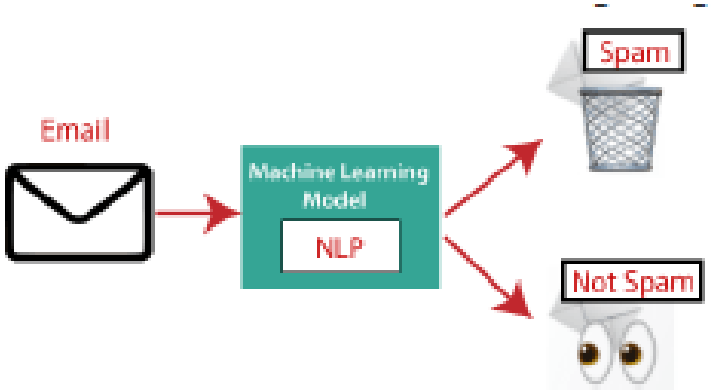
1. Question Answering



Question Answering focuses on building systems that automatically answer the questions asked by humans in a natural language.

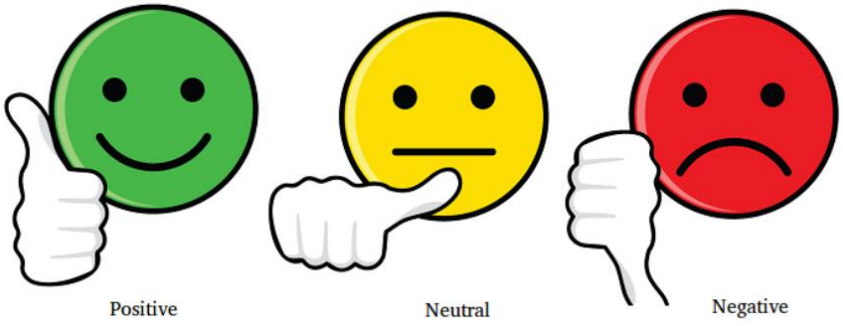
2. Spam Detection

Spam detection is used to detect unwanted e-mails getting to a user's inbox.



3. Sentiment Analysis

Sentiment Analysis is also known as **opinion mining**. It is used on the web to analyse the attitude, behaviour, and emotional state of the sender. This application is implemented through a combination of NLP (Natural Language Processing) and statistics by assigning the values to the text (positive, negative, or natural), identify the mood of the context (happy, sad, angry, etc.)



4. Machine Translation

Machine translation is used to translate text or speech from one natural language to another natural language.

Example: Google Translator

5. Spelling correction

Microsoft Corporation provides word processor software like MS-word, PowerPoint for the spelling correction.

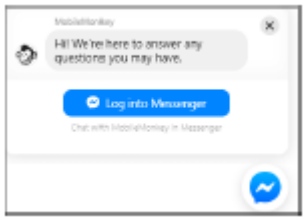


6. Speech Recognition

Speech recognition is used for converting spoken words into text. It is used in applications, such as mobile, home automation, video recovery, dictating to Microsoft Word, voice biometrics, voice user interface, and so on.

7. Chatbot

Implementing the Chatbot is one of the important applications of NLP. It is used by many companies to provide the customer's chat services.



8. Information extraction

Information extraction is one of the most important applications of NLP. It is used for extracting structured information from unstructured or semi-structured machine-readable documents.

9. Natural Language Understanding (NLU)

It converts a large set of text into more formal representations such as first-order logic structures that are easier for the computer programs to manipulate notations of the natural language processing.

NLP Pipeline

The natural language processing (NLP) pipeline refers to the sequence of processes involved in analyzing and understanding human language. The following is a typical NLP pipeline:

Step1: Sentence Segmentation

Sentence Segment is the first step for building the NLP pipeline. It breaks the paragraph into separate sentences.

Example: Consider the following paragraph -

Independence Day is one of the important festivals for every Indian citizen. It is celebrated on the 15th of August each year ever since India got independence from the British rule. The day celebrates independence in the true sense.

Sentence Segment produces the following result:

1. "Independence Day is one of the important festivals for every Indian citizen."

2. "It is celebrated on the 15th of August each year ever since India got independence from the British rule."

3. "This day celebrates independence in the true sense."

Step2: Word Tokenization

Word Tokenizer is used to break the sentence into separate words or tokens.

Example:

Naresh Technologies offers Corporate Training, Summer Training, Online Training, and Winter Training.

Word Tokenizer generates the following result:

"Naresh", "Technologies", "offers", "Corporate", "Training", "Summer", "Training", "Online", "Training", "and", "Winter", "Training", "."

Step3: Stemming

Stemming is used to normalize words into its base form or root form. For example, celebrates, celebrated and celebrating, all these words are originated with a single root word "celebrate." The big problem with stemming is that sometimes it produces the root word which may not have any meaning.

For Example, intelligence, intelligent, and intelligently, all these words are originated with a single root word "intelligen." In English, the word "intelligen" do not have any meaning.

Step 4: Lemmatization

Lemmatization is quite like the Stemming. It is used to group different inflected forms of the word, called Lemma. The main difference between Stemming and lemmatization is that it produces the root word, which has a meaning.

For example: In lemmatization, the words intelligence, intelligent, and intelligently has a root word intelligent, which has a meaning.

Step 5: Identifying Stop Words

In English, there are a lot of words that appear very frequently like "is", "and", "the", and "a". NLP pipelines will flag these words as stop words. **Stop words** might be filtered out before doing any statistical analysis.

Example: He **is a** good boy.

Step 6: Dependency Parsing

Dependency Parsing is used to find that how all the words in the sentence are related to each other.

Step 7: POS tags

POS stands for parts of speech, which includes Noun, verb, adverb, and Adjective. It indicates that how a word functions with its meaning as well as grammatically within the sentences. A word has one or more parts of speech based on the context in which it is used.

Example: "Google" something on the Internet.

In the above example, Google is used as a verb, although it is a proper noun.

Step 8: Named Entity Recognition (NER)

Named Entity Recognition (NER) is the process of detecting the named entity such as person name, movie name, organization name, or location.

Example: Steve Jobs introduced iPhone at the Macworld Conference in San Francisco, California.

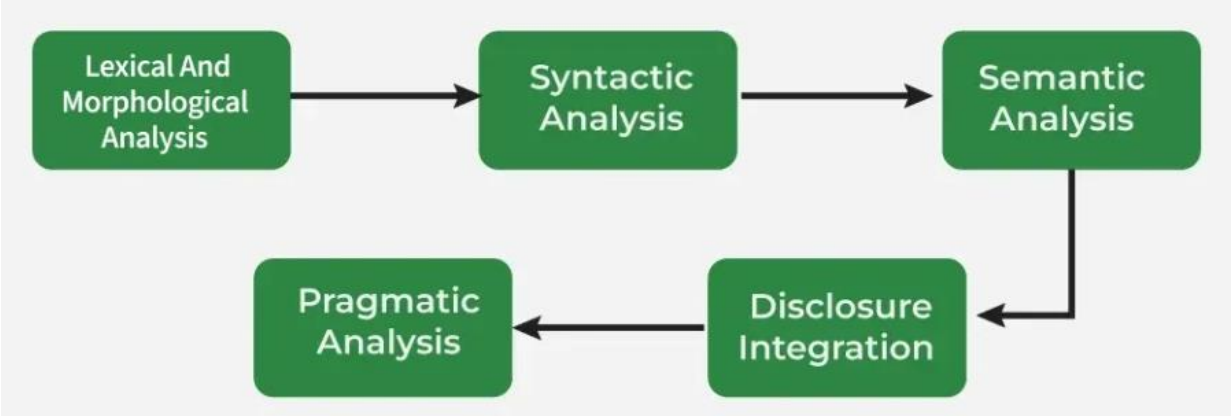
Step 9: Chunking

Chunking is used to collect the individual piece of information and grouping them into bigger pieces of sentences.

Example: the digits on the credit card.

Phases of Natural Language Processing

Natural Language Processing (NLP) helps computers to understand, analyze and interact with human language. It involves a series of phases that work together to process language, and each phase helps in understanding structure and meaning of human language.



1. Lexical and Morphological Analysis

Lexical Analysis

It focuses on identifying and processing words (or lexemes) in a text. It breaks down the input text into individual tokens that are meaningful units of language such as words or phrases.

Key tasks in Lexical analysis:

1. **Tokenization**: Process of dividing a text into smaller chunks called tokens. For example, the sentence "I love programming" would be tokenized into ["I", "love", "programming"].
2. **Part-of-Speech Tagging:** Assigning parts of speech such as noun, verb, adjective to each token in the sentence. This helps us to understand grammatical roles of words in the context.

Example: Consider the sentence: "I am reading a book."

- Tokenization: Sentence is broken down into individual tokens or words: ["I", "am", "reading", "a", "book"]
- Part-of-Speech Tagging: Each token is assigned a part of speech: ["I" → Pronoun (PRP), "am" → Verb (VBP), "reading" → Verb (VBG), "a" → Article (DT), "book" → Noun (NN)]

Importance of Lexical Analysis

- **Word Identification:** It breaks text into tokens which helps the system to understand individual words for further processing.
- **Text Simplification:** It simplifies text through tokenization and stemming which improves accuracy in NLP tasks.

Morphological Analysis

It deals with **morphemes** which are the smallest units of meaning in a word. It is important for understanding structure of words and their parts by identifying free morphemes (independent words like "cat") and bound morphemes (like prefixes or suffixes e.g. "un-" or "-ing").

Key tasks in morphological analysis:

1. **Stemming:** Reducing words to their root form like "running" to "run".
2. **Lemmatization**: Converting words to their base or dictionary form considering the context like "better" becomes "good".

Importance of Morphological Analysis

1. **Understanding Word Structure**: It helps in breaking the composition of complex words.

2. **Improving Accuracy**: It enhances accuracy of tasks such as part-of-speech tagging, syntactic parsing and machine translation.

By identifying and analyzing morphemes system can identify text correctly at the most basic level which helps in more advanced NLP applications.

2. Syntactic Analysis (Parsing)

Syntactic Analysis helps in understanding how words in a sentence are arranged according to grammar rules. It ensures that the sentence follows correct grammar which makes the meaning clearer. The goal is to create a parse tree which is a diagram showing the structure of sentence. It breaks the sentence into parts like the subject, verb and object and shows how these parts are connected. This helps machines understand the relationships between words in the sentence.

Key components of syntactic analysis include:

- **POS Tagging**: Assigning parts of speech (noun, verb, adjective) to words in a sentence as discussed earlier.
- **Ambiguity Resolution**: Handling words that have multiple meanings (e.g "book" can be a noun or a verb).

Examples

Consider the following sentences:

- Correct Syntax: "John eats an apple."
- Incorrect Syntax: "Apple eats John an."

Despite using same words only, the first sentence is grammatically correct and makes sense. The correct arrangement of words according to grammatical rules is what makes the sentence meaningful. By analysing

sentence structure NLP systems can better understand and generate human language. This helps in tasks like machine translation, sentiment analysis and information retrieval by making the text clearer and reducing confusion.

3. Semantic Analysis

Semantic Analysis focuses on understanding meaning behind words and sentences. It ensures that the text is not only grammatically correct but also logically coherent and contextually relevant. It aims to understand dictionary definitions of words and their usage in context and also find whether the arrangement of words in a sentence makes logical sense.

Key Tasks in Semantic Analysis

1. **Named Entity Recognition (NER)**: It identifies and classifies entities such as names of people, locations, organizations, dates and more. These entities provide important meaning in the text and help in understanding the context. For example, in the sentence "Tesla announced its new electric vehicle in California," NER would identify "Tesla" as an organization and "California" as a location.

2. **Word Sense Disambiguation (WSD)**: Many words have multiple meanings depending on the context in which they are used. It identifies the correct meaning of a word based on its surrounding text. For example, word "bank" can refer to a financial institution or the side of a river. It uses context to identify which meaning applies in each sentence which ensures that interpretation is accurate.

Example of Semantic Analysis

"Apple eats a John." while grammatically correct this sentence doesn’t make sense semantically because an apple cannot "eat" a person. Semantic

analysis ensures that the meaning is logically sound and contextually appropriate. It is important for various NLP applications including machine translation, information retrieval and question answering.

4. Discourse Integration

It is the process of understanding how individual sentences or segments of text connect and relate to each other within a broader context. This phase ensures that the meaning of a text is consistent and coherent across multiple sentences or paragraphs. It is important for understanding long or complex texts where meaning focuses on previous statements.

Key aspects of discourse integration:

- **Anaphora Resolution**: Anaphora refers to the use of pronouns or other references that depend on earlier parts of the text. For example in the sentence "Taylor went to the store. She bought groceries" pronoun "She" refers back to "Taylor." It ensures that references like these are correctly understood by linking them to their antecedents.
- **Contextual References**: Many words or phrases can only be fully understood when considered in the context of following sentences. It helps in interpreting how certain words or phrases focuses on context. For example "It was a great day" is clearer when you know what event or situation is being discussed.

Example of Discourse Integration

1. "Taylor went to the store to buy some groceries. She realized she forgot her wallet." Understanding that "Taylor" is the antecedent of "she" is important for understanding sentence's meaning.

2. "This is unfair!" helps in understand what "this" refers to we need to identify following sentences. Without context statement's meaning remains unclear.

It is important for NLP applications like machine translation, chatbots and text summarization. It ensures that meaning remains same across sentences which helps machines to understand context. This enables accurate and natural responses in applications like conversational AI and document translation.

5. Pragmatic Analysis

Pragmatic analysis helps in understanding the deeper meaning behind words and sentences by looking beyond their literal meanings. While semantic analysis looks at the direct meaning it considers the speaker's or writer's intentions, tone and context of the communication.

Key tasks in pragmatic analysis:

- **Understanding Intentions**: Sometimes language doesn't mean what it says literally. For example, when someone asks, "Can you pass the salt?" it's not about ability but a polite request. It helps to understand true intention behind such expressions.
- **Figurative Meaning**: Language often uses idioms or metaphors that can't be taken literally.

Examples of Pragmatic Analysis

- "Hello! What time is it?" here it might be a straightforward request for the current time, but it could also imply concern about being late.
- For example, "I'm falling for you" means "I love you" not literally falling. It helps to interpret these non-literal meanings.

It is important for NLP tasks like sentiment analysis, chatbots and conversation-based AI. It helps machines to understand the speaker's intentions, tone and context which go beyond the literal meaning of words. By identifying sarcasm and emotions this help systems to respond naturally which improves human-computer interaction. By combining these phases NLP systems can effectively interpret, analyze and generate human language making more intelligent and natural interactions between humans and machines.

Why NLP is difficult?

NLP is difficult because Ambiguity and Uncertainty exist in the language.

Ambiguity

There are the following three ambiguity -

Lexical Ambiguity:

Lexical Ambiguity exists in the presence of two or more possible meanings of the sentence within a single word.

Example:

Manya is looking for a **match**.

In the above example, the word match refers to that either Manya is looking for a partner or Manya is looking for a match. (Cricket or other match)

Syntactic Ambiguity

Syntactic Ambiguity exists in the presence of two or more possible meanings within the sentence.

Example: I saw the girl with the binocular.

In the above example, did I have the binoculars? Or did the girl have the binoculars?

Referential Ambiguity

Referential Ambiguity exists when you are referring to something using the pronoun.

Example: Kiran went to Sunita. She said, "I am hungry."

In the above sentence, you do not know that who is hungry, either Kiran or Sunita.

NLP APIs

Natural Language Processing APIs allow developers to integrate human-to-machine communications and complete several useful tasks such as speech recognition, chatbots, spelling correction, sentiment analysis, etc.

A list of NLPS APIs is given below:

IBM Watson API

IBM Watson API combines different sophisticated machine learning techniques to enable developers to classify text into various custom categories. It supports multiple languages, such as English, French, Spanish, German, Chinese, etc. With the help of IBM Watson API, you can extract insights from texts, add automation in workflows, enhance search, and understand the sentiment. The main advantage of this API is that it is very easy to use.

Pricing:

Firstly, it offers a free 30-day trial IBM cloud account. You can also opt for its paid plans.

Chatbot API

Chatbot API allows you to create intelligent chatbots for any service. It supports Unicode characters, classifies text, multiple languages, etc. It is very easy to use. It helps you to create a chatbot for your web applications.

Pricing: Chatbot API is free for 150 requests per month. You can also opt for its paid version, which starts from $100 to $5,000 per month.

Speech to text API

Speech to text API is used to convert speech to text

Pricing: Speech to text API is free for converting 60 minutes per month. Its paid version starts from $500 to $1,500 per month.

Sentiment Analysis API

Sentiment Analysis API is also called as '**opinion mining**' which is used to identify the tone of a user (positive, negative, or neutral)

Pricing: Sentiment Analysis API is free for less than 500 requests per month. Its paid version starts from $19 to $99 per month.

Translation API by SYSTRAN

The Translation API by SYSTRAN is used to translate the text from the source language to the target language. You can use its NLP APIs for language detection, text segmentation, named entity recognition, tokenization, and many other tasks.

Pricing: This API is available for free. But for commercial users, you need to use its paid version.

Text Analysis API by AYLIEN

Text Analysis API by AYLIEN is used to derive meaning and insights from the textual content. It is available for both free as well as paid from$119 per month. It is easy to use. **Pricing:** This API is available free for 1,000 hits per day. You can also use its paid version, which starts from $199 to S1, 399 per month.

Cloud NLP API

The Cloud NLP API is used to improve the capabilities of the application using natural language processing technology. It allows you to carry various natural language processing functions like sentiment analysis and language detection. It is easy to use.

Pricing: Cloud NLP API is available for free.

Google Cloud Natural Language API

Google Cloud Natural Language API allows you to extract beneficial insights from unstructured text. This API allows you to perform entity recognition, sentiment analysis, content classification, and syntax analysis in more the 700 predefined categories. It also allows you to perform text analysis in multiple languages such as English, French, Chinese, and German.

Pricing: After performing entity analysis for 5,000 to 10,000,000 units, you need to pay $1.00 per 1000 units per month.

NLP Libraries:

Scikit-learn: It provides a wide range of algorithms for building machine learning models in Python.

Natural language Toolkit (NLTK): NLTK is a complete toolkit for all NLP techniques.

Pattern: It is a web mining module for NLP and machine learning.

Text Blob: It provides an easy interface to learn basic NLP tasks like sentiment analysis, noun phrase extraction, or pos-tagging.

Quepy: Quepy is used to transform natural language questions into queries in a database query language.

SpaCy: SpaCy is an open-source NLP library which is used for Data Extraction, Data Analysis, Sentiment Analysis, and Text Summarization.

Gensim: Gensim works with large datasets and processes data streams.

Difference between Natural language and Computer Language

Natural Language	Computer Language
Natural language has a very large vocabulary.	Computer language has a very limited vocabulary.
Natural language is easily understood by humans.	Computer language is easily understood by the machines.
Natural language is ambiguous in nature.	Computer language is unambiguous.

UNIT-I

Finding the Structure of Words: Words and Their Components, Issues and Challenges, Morphological Models

Finding the Structure of Documents: Introduction, Methods, Complexity of the Approaches, Performances of the Approaches, Features

Finding the Structure of Words

Language is complex, structured, and requires an understanding of various components like morphology, syntax, phonology, and semantics. Each discipline focuses on different aspects of language, such as word forms, sentence structure, pronunciation, and the meaning of words. Understanding these components is crucial for language processing, especially in multilingual contexts.

Morphological parsing plays a significant role in identifying the types of words and their structures in relation to grammatical properties. The difficulty of parsing can vary; some languages make word boundaries clear with whitespace, while others require readers to determine them within diverse phonological forms.

In natural language processing (NLP), finding the structure of words involves breaking down words into their constituent parts and identifying the relationships between those parts. This process is known as morphological analysis, and it helps NLP systems understand the structure of language.

There are several ways to find the structure of words in NLP, including:

1. Tokenization: This involves breaking a sentence or document into individual words or tokens, which can then be analyzed further.

2. Stemming and Lemmatization: These techniques involve reducing words to their base or root form, which can help identify patterns and relationships between words.

3. Part-of-Speech Tagging: This involves labelling each word in a sentence with its part of speech, such as noun, verb, adjective, or adverb.

4. Parsing: This involves analyzing the grammatical structure of a sentence by identifying its constituent parts, such as subject, object, and predicate.

5. Named Entity Recognition: This involves identifying and classifying named entities in text, such as people, organizations, and locations.

6. Dependency Parsing: This involves analyzing the relationships between words in a sentence and identifying which words depend on or modify other words.

By finding the structure of words in text, NLP systems can perform a wide range of tasks, such as machine translation, text classification, sentiment analysis, and information extraction.

****

Words and Their Components

Words are defined in most languages as the smallest linguistic units that can form a complete utterance by themselves. The minimal parts of words that deliver pieces of meaning to them are called morphemes. Depending on the means of communication, morphemes are spelled out via graphemes- symbols of writing such as letters or characters or are realized through phonemes, the distinctive units of sound in spoken language.

The analysis of words and their components is important for various NLP tasks such as stemming, lemmatization, part-of-speech tagging, and sentiment analysis.

Tokens:

In natural language processing (NLP), a token refers to a sequence of characters that represents a meaningful unit of text. This could be a word, punctuation mark, number, or other entity that serves as a basic unit of analysis in NLP.

For example, in the sentence "The quick brown fox jumps over the lazy dog," the tokens are "The," "quick," "brown," "fox," "jumps," "over," "the," "lazy," and "dog." Each of these tokens represents a separate unit of meaning that can be analyzed and processed by an NLP system.

Here are some additional examples of tokens:

- **Punctuation marks**, such as periods, commas, and semicolons, are tokens that represent the boundaries between sentences and clauses.
- **Numbers**, such as "123" or "3.14," are tokens that represent numeric quantities or measurements.
- **Special characters**, such as "@" or "#" can be tokens that represent symbols used in social media or other online contexts.

Tokens are often used as the input for various NLP tasks, such as text classification, sentiment analysis, and named entity recognition. In these tasks, the NLP system analyzes the tokens to identify patterns and relationships between them and uses this information to make predictions or draw insights about the text.

To analyze and process text effectively, NLP systems must be able to identify and distinguish between different types of tokens and understand

their relationships with one another. This can involve tasks such as tokenization, where the text is divided into individual tokens, and part-of-speech tagging, where each token is assigned a grammatical category (such as noun, verb, or adjective). By accurately identifying and processing tokens, NLP systems can better understand the meaning and structure of a text.

Lexemes:

In natural language processing (NLP), a lexeme is a unit of vocabulary that represents a single concept, regardless of its inflected forms or grammatical variations. It can be thought of as the abstract representation of a word, with all its possible inflections and variations.

For example, the word "run" has many inflected forms, such as "ran," "running," and "runs." These inflections are not considered separate lexemes because they all represent the same concept of running or moving quickly on foot.

In contrast, words that have different meanings, even if they are spelled the same way, are considered separate lexemes. For example, the word "bank" can refer to a financial institution or the edge of a river. These different meanings are considered separate lexemes because they represent different concepts.

Here are some additional examples of lexemes

- "Walk" and "walked" are inflected forms of the same lexeme, representing the concept of walking.
- "Cat" and "cats" are inflected forms of the same lexeme, representing the concept of a feline animal.
- "Bank" and "banking" are derived forms of the same lexeme, representing the concept of finance and financial institutions.

Lexical analysis involves identifying and categorizing lexemes in a text, which is an important step in many NLP tasks, such as text classification, sentiment analysis, and information retrieval. By identifying and categorizing lexemes, NLP systems can better understand the meaning and context of a text.

Lexical analysis is also used to identify and analyze the morphological and syntactical features of a word, such as its part of speech, inflection, and derivation.

This information is important for tasks such as stemming, lemmatization, and part-of-speech tagging, which involve reducing words to their base or root forms and identifying their grammatical functions.

Morphemes:

In natural language processing (NLP), morphemes are the smallest units of meaning in a language. A morpheme is a sequence of phonemes (the smallest units of sound in a language) that carries meaning.

Morphemes can be divided into two types:

free morphemes and bound morphemes.

Free morphemes are words that can stand alone and convey meaning. Examples of free morphemes include "book," "cat," "happy," and "run."

Bound morphemes are units of meaning that cannot stand alone but must be attached to a free morpheme to convey meaning. Bound morphemes can be further divided into two types: **prefixes and suffixes**.

- A prefix is a bound morpheme that is added to the beginning of a word to change its meaning. For example, the prefix "un-" added to the word "happy" creates the word "unhappy," which means not happy.

- A suffix is a bound morpheme that is added to the end of a word to change its meaning. For example, the suffix "-ed" added to the word "walk" creates the word "walked," which represents the past tense of "walk."

Here are some examples of words broken down into their morphemes:

- "unhappily" = "un-" (prefix meaning "not") + "happy" + "-ly" (suffix meaning "in a manner of")
- "rearrangement" = "re-" (prefix meaning "again") + "arrange" + "-ment" (suffix indicating the act of doing something)
- "cats" = "cat" (free morpheme) + "-s" (suffix indicating plural form)

By analyzing the morphemes in a word, NLP systems can better understand its meaning and how it relates to other words in a sentence. This can be helpful for tasks such as part-of-speech tagging, sentiment analysis, and language translation.

Typology:

In natural language processing (NLP), typology refers to the classification of languages based on their structural and functional features. This can include features such as word order, morphology, tense and aspect systems, and syntactic structures.

There are many different approaches to typology in NLP, but a common one is the distinction between analytic and synthetic languages.

Analytic languages have a relatively simple grammatical structure and tend to rely on word order and prepositions to convey meaning.

In contrast, **synthetic languages** have a more complex grammatical structure and use inflections and conjugations to indicate tense, number, and other grammatical features.

Agglutinative languages have morphemes associated with only a single function at a time (as in Korean, Japanese, Finnish, and Tamil, etc.).

Fusional languages are defined by their feature-per-morpheme ratio higher than one (as in Arabic, Czech, Latin, Sanskrit, German, etc.).

In accordance with the notions about word formation processes mentioned earlier, we can also discern:

Concatenative languages linking morphs and morphemes one after another.

Nonlinear languages allow structural components to merge consequentially to apply tonal morphemes or change the consonantal or vocalic templates of words.

****

Issues and Challenges

Finding the structure of words in natural language processing (NLP) can be a challenging task due to various issues and challenges. Some of these issues and challenges are:

Irregularity:

Irregularity is a challenge in natural language processing (NLP) because it refers to words that do not follow regular patterns of formation or inflection. Many languages have irregular words that are exceptions to the standard rules, making it difficult for NLP systems to accurately identify and categorize these words.

For example, in English, irregular verbs such as "go," "do," and "have" do not follow the regular pattern of adding "-ed" to the base form to form the

past tense. Instead, they have their unique past tense forms ("went," "did," "had") that must be memorized.

Similarly, in English, there are many irregular plural nouns, such as "child" and "foot," that do not follow the standard rule of adding "-s" to form the plural. Instead, these words have their unique plural forms ("children," "feet") that must be memorized.

Irregularity can also occur in inflectional morphology, where different forms of a word are created by adding inflectional affixes. For example, in Spanish, the irregular verb "tener" (to have) has a unique conjugation pattern that does not follow the standard pattern of other regular verbs in the language.

To address the challenge of irregularity in NLP, researchers have developed various techniques, including creating rule-based systems that incorporate irregular forms into the standard patterns of word formation or using machine learning algorithms that can learn to recognize and categorize irregular forms based on the patterns present in large datasets.

However, dealing with irregularity remains an ongoing challenge in NLP, particularly in languages with a high degree of lexical variation and complex morphological systems. Therefore, NLP researchers are continually working to improve the accuracy of NLP systems in dealing with irregularity.

Ambiguity:

Ambiguity is a challenge in natural language processing (NLP) because it refers to situations where a word or phrase can have multiple possible meanings, making it difficult for NLP systems to accurately identify the intended meaning. Ambiguity can arise in various forms, such as homonyms, polysemous words, and syntactic ambiguity.

Homonyms are words that have the same spelling and pronunciation but different meanings. For example, the word "bank" can refer to a financial institution or the side of a river. This can create ambiguity in NLP tasks, such as named entity recognition, where the system needs to identify the correct entity based on the context.

Polysemous words are words that have multiple related meanings. For example, the word "book" can refer to a physical object or the act of reserving something. In this case, the intended meaning of the word can be difficult to identify without considering the context in which the word is used.

Syntactic ambiguity occurs when a sentence can be parsed in multiple ways. For example, the sentence "I saw her duck" can be interpreted as "I saw the bird she owns" or "I saw her lower her head to avoid something." In this case, the meaning of the sentence can only be determined by considering the context in which it is used.

Ambiguity can also occur due to cultural or linguistic differences. For example, the phrase "kick the bucket" means "to die" in English, but its meaning may not be apparent to non-native speakers or speakers of other languages.

To address ambiguity in NLP, researchers have developed various techniques, including using contextual information, part-of-speech tagging, and syntactic parsing to disambiguate words and phrases. These techniques involve analyzing the surrounding context of a word to determine its intended meaning based on the context. Additionally, machine learning algorithms can be trained on large datasets to learn to disambiguate words and phrases automatically. However, dealing with ambiguity remains an ongoing challenge in NLP, particularly in languages with complex grammatical structures and a high degree of lexical variation.

Productivity:

Productivity is a challenge in natural language processing (NLP) because it refers to the ability of a language to generate new words or forms based on existing patterns or rules. This can create a vast number of possible word forms that may not be present in dictionaries or training data, which makes it difficult for NLP systems to accurately identify and categorize words.

For example, in English, new words can be created by combining existing words, such as "smartphone," "cyberbully," or "workaholic." These words are formed by combining two or more words to create a new word with a specific meaning.

Another example is the use of prefixes and suffixes to create new words. For instance, in English, the prefix "un-" can be added to words to create their opposite meaning, such as "happy" and "unhappy." The suffix "-er" can be added to a verb to create a noun indicating the person who performs the action, such as "run" and "runner."

Productivity can also occur in inflectional morphology, where different forms of a word are created by adding inflectional affixes. For example, in English, the verb "walk" can be inflected to "walked" to indicate the past tense. Similarly, the adjective "big" can be inflected to "bigger" to indicate a comparative degree.

These examples demonstrate how productivity can create a vast number of possible word forms, making it challenging for NLP systems to accurately identify and categorize words. To address this challenge, NLP researchers have developed various techniques, including morphological analysis algorithms that use statistical models to predict the likely structure of a word based on its context. Additionally, machine learning algorithms can be

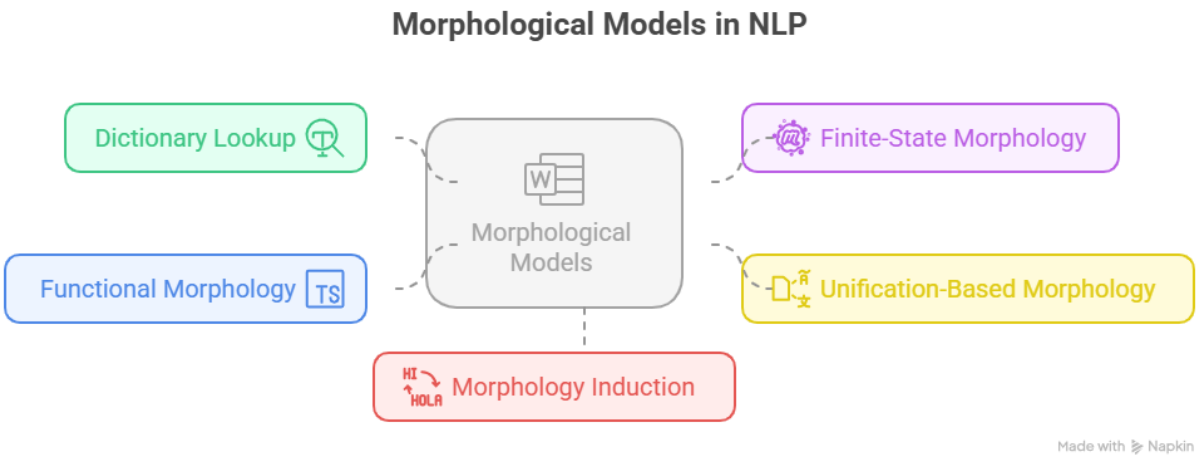
trained on large datasets to learn to recognize and categorize new word forms.

****

Morphological Models

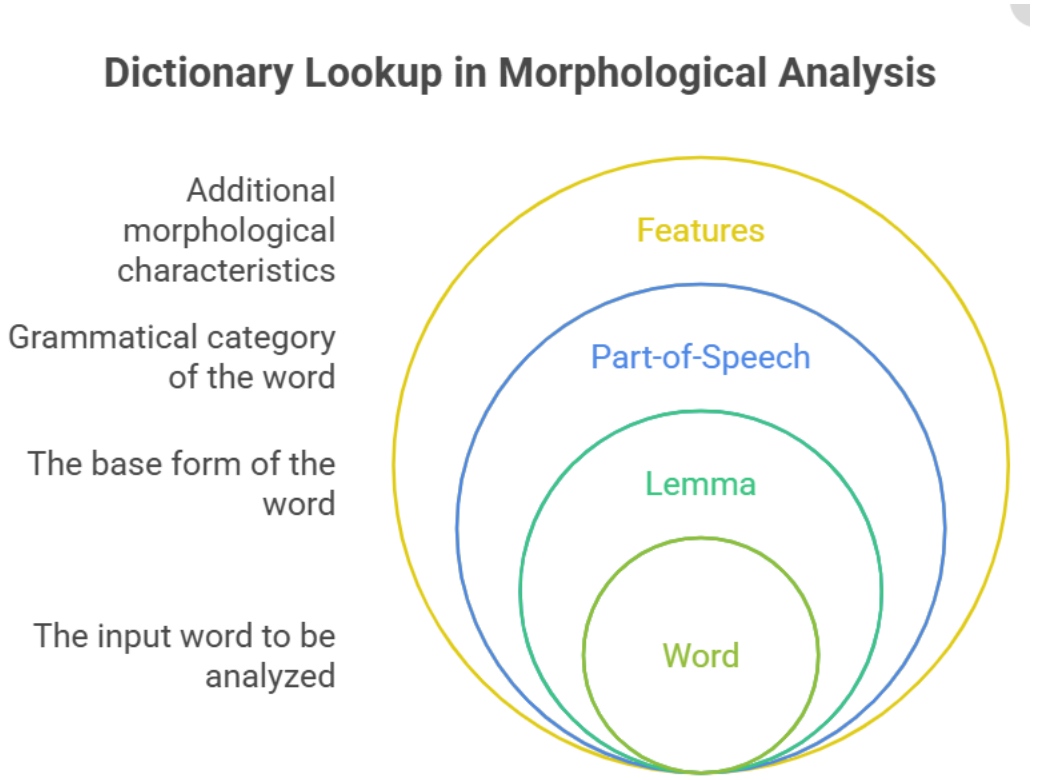
a comprehensive overview of various morphological models used in natural language processing (NLP). It explores dictionary lookup, finite-state morphology, unification-based morphology, functional morphology, and morphology induction, illustrating each with relevant examples.

The aim is to provide a clear understanding of the strengths and weaknesses of each approach in handling the complexities of word formation.



1. Dictionary Lookup:

Dictionary lookup is the simplest approach to morphological analysis. It involves storing all possible word forms in a dictionary along with their corresponding morphological information (lemma, part-of-speech, features). When analysing a word, the system simply looks it up in the dictionary.



Example:

Let's say we have a dictionary containing the following entries:

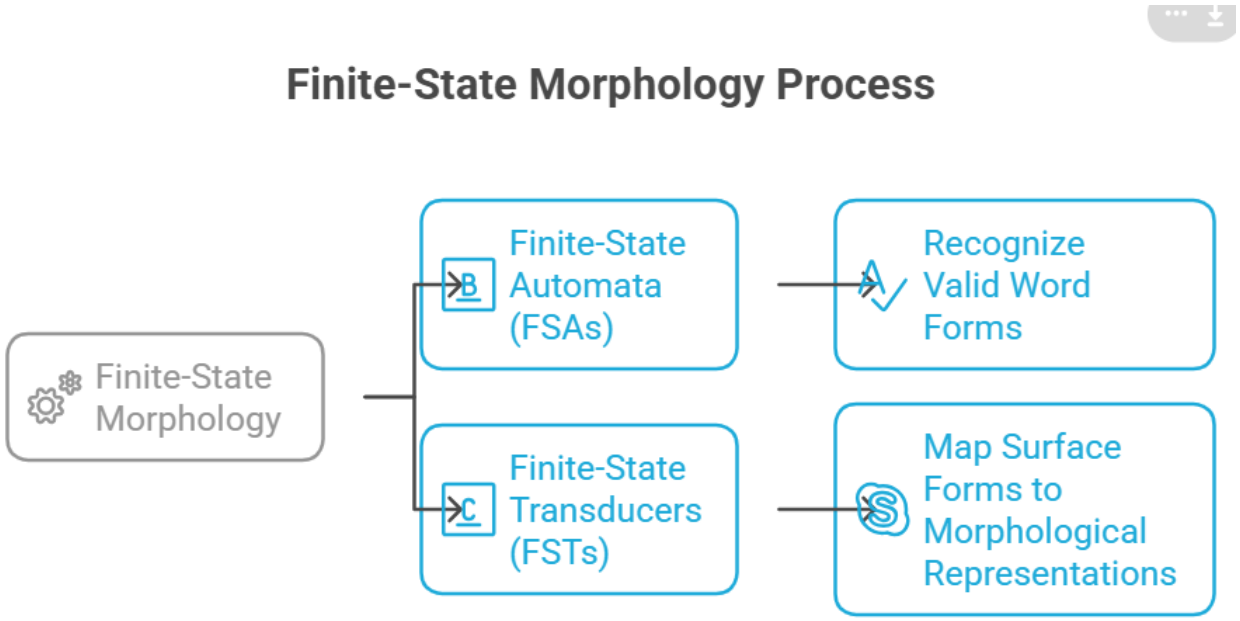
- *walk*: Verb, Lemma: walk, Tense: Present, Number: Singular
- *walks*: Verb, Lemma: walk, Tense: Present, Number: Singular, Person: 3rd
- *walked*: Verb, Lemma: walk, Tense: Past
- *walking*: Verb, Lemma: walk, Tense: Present Participle

If the system encounters the word "walked," it looks it up in the dictionary and retrieves the information that it is the past tense form of the verb "walk."

2. Finite-State Morphology:

Finite-state morphology (FSM) uses finite-state automata (FSAs) and finite-state transducers (FSTs) to model morphological processes. FSAs recognize

valid word forms, while FSTs map between surface forms and their underlying morphological representations.



Example:

Consider the English pluralization rule for nouns. We can represent this with an FST.

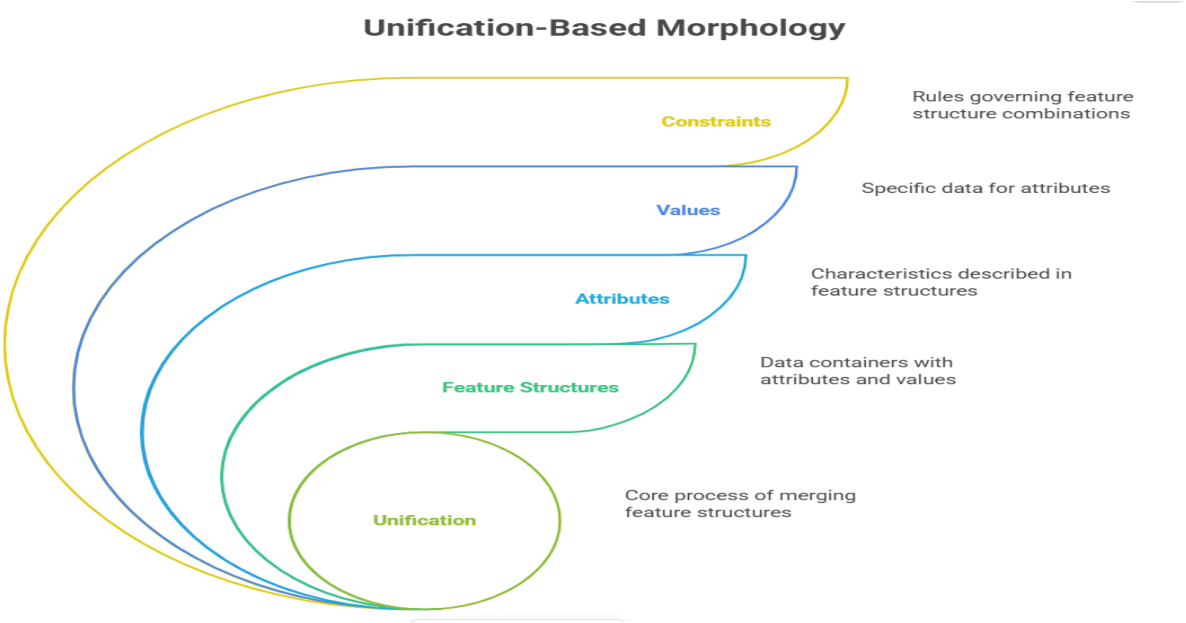
- **Input Alphabet:** Letters of the alphabet (a-z)
- **Output Alphabet:** Letters of the alphabet (a-z) and morphological tags (+PL)

The FST would have states representing different stages of processing. For regular nouns, it would simply copy the input to the output until the end of the word, then add "+PL" and the letter "s". For nouns ending in "y," it would change the "y" to "i" before adding "+es".
For example:

- *cat -> cat+PLs*
- *fly -> fli+Ples*

3. Unification-Based Morphology:

Unification-based morphology represents morphological information using feature structures. These feature structures contain attributes and their corresponding values. Morphological rules are expressed as constraints on these feature structures. Unification is the process of combining two feature structures by merging compatible information.



Example:

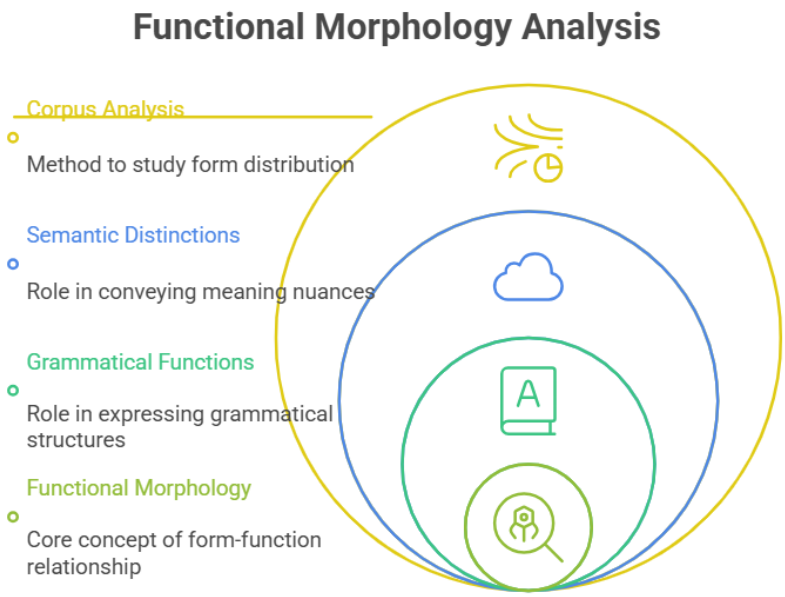
Consider the word "eats." We can represent its morphological information using the following feature structure:

```
[
  LEMMA: eat,
  CATEGORY: verb,
  TENSE: present,
  NUMBER: singular,
  PERSON: 3rd
]
```

A morphological rule for 3rd person singular present tense verbs might specify that the suffix "-s" is added to the lemma and the features NUMBER and PERSON are set to singular and 3rd, respectively. Unification would combine the feature structure of the lemma "eat" with the feature structure derived from the suffix "-s" to produce the feature structure above.

4. Functional Morphology

Functional morphology focuses on the relationship between form and function in morphology. It emphasizes the role of morphological processes in expressing grammatical functions and semantic distinctions. It often involves analysing the frequency and distribution of different morphological forms in a corpus to understand their functional roles.



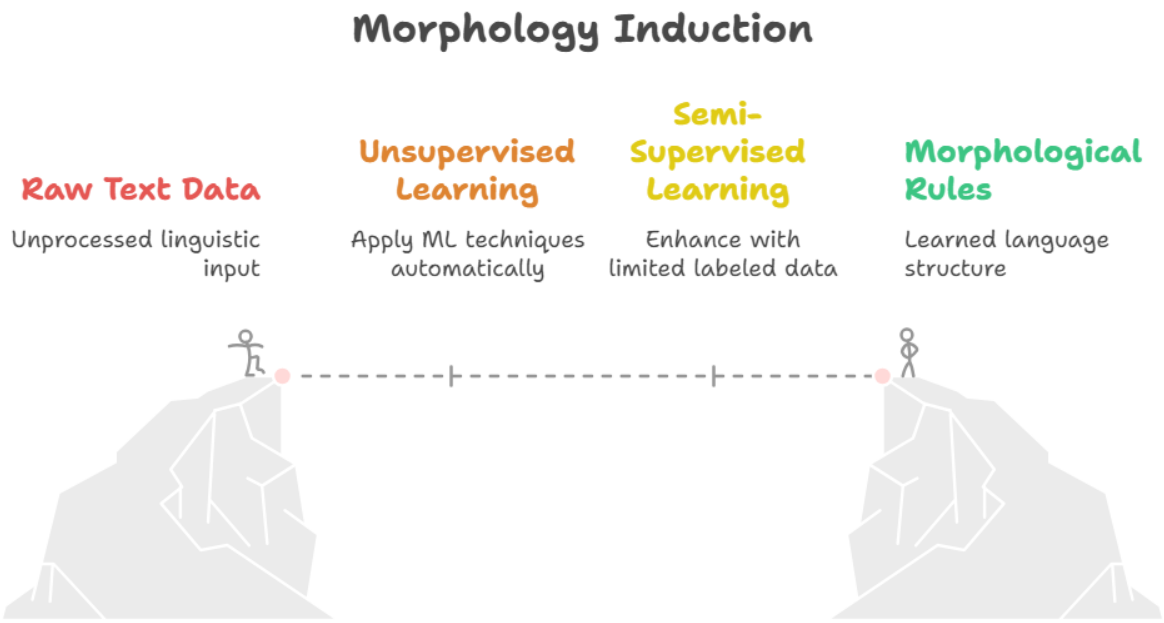
Example:

Consider the use of different plural markers in English. While "-s" is the most common plural marker, other markers like "-en" (children, oxen) and irregular forms (mice, feet) exist. Functional morphology would analyze the factors that influence the choice of a particular plural marker, such as historical origin, frequency of use, and semantic category of the noun. For

instance, the irregular plurals often belong to older words or words with specific semantic properties.

5. Morphology Induction:

Morphology induction is the task of automatically learning morphological rules from raw text data without any prior knowledge of the language's morphology. This is typically done using unsupervised or semi-supervised machine learning techniques.



Example:

Given a corpus of English text, a morphology induction system might identify that the suffix "-ed" is often attached to verbs and that the resulting words often appear in past tense contexts. It might also learn that the suffix "-ing" is attached to verbs to form present participles. The system would then group words with similar suffixes and contexts into morphological paradigms.

Techniques used in morphology induction:

- **Minimum Description Length (MDL):** Finds the morphological analysis that minimizes the combined length of the lexicon and the rules needed to generate the observed word forms.
- **Probabilistic Models:** Use probabilistic models to estimate the likelihood of different morphological analyses.
- **Clustering:** Groups words with similar prefixes, suffixes, and contexts into morphological classes.

****

Differentiate between derivational and inflectional morphemes. provide examples to illustrate their roles in word structure.

Derivational morphemes change the meaning or grammatical category of a word, while inflectional morphemes only modify its grammatical form without creating a new word.

Derivational Morphemes

Derivational morphemes are prefixes or suffixes that form new words or change a word's part of speech. For example:

- "happy" + "un-" = "unhappy" (changes meaning)
- "teach" + "-er" = "teacher" (verb to noun)
- "love" + "-ly" = "lovely" (noun to adjective)
- "friend" + "-less" = "friendless" (meaning change) These morphemes may change a word's class (noun to adjective, verb to noun, etc.) or just its meaning.

Inflectional Morphemes

Inflectional morphemes are suffixes that convey grammatical information—such as tense, number, possession, or comparison—without altering the word's meaning or class. Examples include:

- "cat" + "-s" = "cats" (plural)
- "mark" + "-ed" = "marked" (past tense)
- "Alex" + "-'s" = "Alex's" (possessive)
- "tall" + "-er" = "taller" (comparative; note: "-er" can also be derivational)

  Inflectional morphemes never result in a new word, always maintain the word's part of speech, and only appear as suffixes in English.

Structural Roles in Words

- Derivational morphemes create complex words and always precede inflectional morphemes within the structure (e.g., "readers": read + -er [derivational] + -s [inflectional]).
- Inflectional morphemes are limited in number (e.g., -s, -ed, -ing) and appear at the edges of word structure, marking grammatical relationships.

In summary, derivational morphemes create new words or meanings, often changing the part of speech, while inflectional morphemes modify words to express grammatical relationships, without changing their core meaning or word class.

Finding the Structure of Documents

Introduction:

In human communication, words and sentences are not arranged randomly; they typically follow a specific structure. For instance, sentences are created by combining words into meaningful grammatical units, which can include statements, requests, and commands. Similarly, in written form, sentences come together to create paragraph-organized units of discourse that focus on a specific point or idea. Additionally, sentences can be connected to one another through explicit discourse markers, such as "therefore."

The automatic extraction of document structures facilitates various subsequent natural language processing tasks. For instance, tasks such as parsing, machine translation, and semantic role labeling utilize sentences as their fundamental processing units.

Sentence boundary annotation is also important for aiding human readability of the output of automatic speech recognition (ASR) systems Furthermore, chunking the input text or speech into topically coherent blocks provides better organization and indexing of the data.

The process of identifying where sentences begin and end within a sequence of characters is termed sentence boundary detection. Likewise, we describe topic segmentation as the process of recognizing the beginning and end of a topic within a series of sentences.

Sentence Boundary Detection:

Sentence boundary detection is a subtask of finding the structure of documents in NLP that involves identifying the boundaries between sentences in a document. This is an important task, as it is a fundamental step in many NLP applications, such as machine translation, text summarization, and information retrieval.

Sentence boundary detection is a challenging task due to the presence of ambiguities and irregularities in natural language, such as abbreviations, acronyms, and names that end with a period. To address these challenges, several methods and techniques have been developed for sentence boundary detection, including:

1. Regular expressions: These are patterns that can be used to match specific character sequences in a text, such as periods followed by whitespace characters, and can be used to identify the end of a sentence.

2. Hidden Markov Models: These are statistical models that can be used to identify the most likely sequence of sentence boundaries in a text, based on the probabilities of different sentence boundary markers.

3. Deep learning models: These are neural network models that can learn complex patterns and features of sentence boundaries from a large corpus of text and can be used to achieve state-of-the-art performance in sentence boundary detection.

Sentence boundary detection is an essential step in many NLP tasks, as it provides the foundation for analysing and interpreting the structure and meaning of a document. By accurately identifying the boundaries between sentences, NLP systems can more effectively extract information, generate summaries, and perform other language-related tasks.

Topic Boundary Detection:

Topic boundary detection is another important subtask of finding the structure of documents in NLP. It involves identifying the points in a document where the topic or theme of the text shifts. This task is particularly useful for organizing and summarizing large amounts of text, as it allows for the identification of different topics or subtopics within a document.

Topic boundary detection is a challenging task, as it involves understanding the underlying semantic structure and meaning of the text, rather than simply identifying specific markers or patterns. As such, there are several methods and techniques that have been developed to address this challenge, including:

1. **Latent Dirichlet Allocation (LDA):** This is a probabilistic topic modeling technique that can be used to identify topics within a corpus of text. By analysing the distribution of words within a text, LDA can identify the most likely topics and subtopics within the text and can be used to locate topic boundaries.

2. TextTiling: This is a technique that involves breaking a text into smaller segments, or "tiles", based on the frequency and distribution of key words and phrases. By comparing the tiles to each other, it is possible to identify shifts in topic or subtopic and locate potential topic boundaries.

3. Coh-Metrix: This is a text analysis tool that uses a range of linguistic and discourse-based features to identify different aspects of text complexity, including topic boundaries. By analysing the patterns of words, syntax, and discourse in a text, Coh-Metrix can identify potential topic boundaries, as well as provide insights into the overall structure and organization of the text.

Topic boundary detection is an important task in NLP, as it enables more effective organization and analysis of large amounts of text. By accurately identifying topic boundaries, NLP systems can more effectively extract and summarize information, identify key themes and ideas, and provide more insightful and relevant responses to user queries.

Methods:

There are several methods and techniques used in NLP to find the structure of documents. Which include

Generative Sequence Classification Methods:

Generative sequence classification methods are a type of NLP method used to find the structure of documents. These methods involve using probabilistic models to classify sequences of words into predefined categories or labels.

One popular generative sequence classification method is Hidden Markov Models (HMMs). HMMs are statistical models that can be used to classify sequences of words by modeling the probability distribution of the observed words given a set of hidden states. The hidden states in an HMM can represent different linguistic features, such as part-of-speech tags or named entities, and the model can be trained using labelled data to learn the most likely sequence of hidden states for a given sequence of words.

Another type of generative sequence classification method is Conditional Random Fields (CRFs). CRFs are similar to HMMs in that they model the conditional probability of a sequence of labels given a sequence of words, but they are more flexible in that they can consider more complex features and dependencies between labels.

Both HMMs and CRFs can be used for tasks like part-of-speech tagging, named entity recognition, and chunking, which involve classifying sequences of words into predefined categories or labels. These methods have been shown to be effective in a variety of NLP applications and are widely used in industry and academia.

Discriminative Sequence Classification Methods:

Discriminative sequence classification methods are another type of NLP method used to find the structure of documents. These methods involve training a model to predict the label or category for a sequence of words in a document, based on the features of the sequence and the context in which it appears.

One popular example of a discriminative sequence classification method is the Maximum Entropy Markov Model (MEMM). MEMMs are a type of discriminative model that can predict the label or category for a sequence of words in a document, based on the features of the sequence and the context in which it appears. MEMMs have been used for tasks such as named entity recognition, part-of-speech tagging, and text classification.

Another example of a discriminative sequence classification method is Conditional Random Fields (CRFs), which were mentioned earlier as a type of generative model. CRFs can also be used as discriminative models, as they can model the conditional probability of a sequence of labels given a sequence of features, without making assumptions about the underlying distribution of the data. CRFs have been used for tasks such as named entity recognition, part-of-speech tagging, and chunking.

Other discriminative sequence classification methods include Hidden Markov Models (HMMs), which were mentioned earlier as a type of generative model. HMMs can also be used as discriminative models, by

directly estimating the probability of a sequence of labels given a sequence of features. HMMs have been used for tasks such as speech recognition, named entity recognition, and part-of-speech tagging.

Hybrid Approaches:

Hybrid approaches to finding the structure of documents in NLP combine multiple methods to achieve better results than any one method alone. For example, a hybrid approach might combine generative and discriminative models or combine different types of models with different types of features.

One example of a hybrid approach is the use of Conditional Random Fields (CRFs) and Support Vector Machines (SVMs) for named entity recognition. CRFs are used to model the dependencies between neighbouring labels in the sequence, while SVMs are used to model the relationship between the input features and the labels.

Another example of a hybrid approach is the use of a rule-based system in combination with machine learning models for sentence boundary detection. The rule-based system might use heuristics to identify common sentence-ending punctuation, while a machine learning model might be trained on a large corpus of text to identify fewer common patterns.

Hybrid approaches can also be used to combine different types of features in a model. For example, a model might use both lexical features (such as the words in the sequence) and syntactic features (such as the part-of-speech tags of the words) to predict the labels for a sequence.

Extensions for Global Modeling for Sentence Segmentation:

Extensions for global modeling for sentence segmentation in NLP involve using algorithms that analyze an entire document or corpus of documents to identify sentence boundaries, rather than analysing sentences in isolation. These methods can be more effective in situations where sentence boundaries are not clearly indicated by punctuation, or where there are other sources of ambiguity.

One example of an extension for global modeling for sentence segmentation is the use of Hidden Markov Models (HMMs). HMMs are statistical models that can be used to identify patterns in a sequence of observations. In the case of sentence segmentation, the observations are the words in the document, and the model tries to identify patterns that correspond to the beginning and end of sentences. HMMs can consider context beyond just the current sentence, which can improve accuracy in cases where sentence boundaries are not clearly marked.

Another example of an extension for global modeling is the use of clustering algorithms. Clustering algorithms group similar sentences together based on features such as the frequency of certain words or the number of common n-grams. Once sentences are clustered together, the boundaries between the clusters can be used to identify sentence boundaries.

Complexity of the Approaches

First, we need to know why it is required. It is required to check that, which approach, or method is better than other one and how accurately predicting the results. So, we need to measure the complexity by this we will tell which approach is better.

The approaches described here have advantages and disadvantages. In each context and under a set of observation features, one approach may be better than another. These approaches can be rated in terms of complexity of their training and prediction algorithms and in terms of their performance on real-world datasets. Some may also require specific preprocessing, such as converting or normalizing continuous features to discreate features.

In terms of complexity, training of discriminative approaches is more complex than training of generative ones because they require multiple passes over the training data to adjust for their features weight. However, generative models such as HELMs can handle multiple orders of magnitude larger training sets and benefit, for instance, from decades of news wire transcripts. On the other hand, they work with only a few features and do not cope well with unseen events.

Compared to local approaches, sequence approaches bring the additional complexity of decoding: finding the best sequence of decision requires evaluating all possible sequences of decisions.

Performances of the Approaches

We can measure the complexity of a method by using performance metrics. In general, the performance of these approaches will depend on factors such as the quality and quantity of the training data, the complexity and variability of the document structure and the specific metrics used to evaluate performance like accuracy, precision, recall, f1-score.

For sentence segmentation in speech or text, performance is usually evaluated using the Error Rate.

Error Rate = Number of errors / Number of examples

Precision = Total Number of Correct Positive Predictions / Total Number of Positive Predictions

Recall=Total Number of Correct Positive Predictions / Total Number of Positive Instances

F1-score=2 * (precision * Recall / precision + Recall)

Various studies have reported different performance levels for these methods. For instance, **rule-based systems** for **sentence boundary detection** have shown error rates as low as 1.41%. **Supervised** classifiers like SVMs, often combined with POS tag features, have achieved F1-measures as high as 97.5%. Hybrid approaches, such as those combining **HMMs with CRFs**, have also demonstrated **high F1-scores, ranging from 78.2% to 89.1**% in different contexts. The specific features used, such as lexical, prosodic, or grammatical features, significantly influence the performance of discriminative approaches.

In conclusion, **finding the structure of documents** through **sentence and topic segmentation** is a complex task addressed by a range of computational methods. From foundational **generative models like**

HMMs to advanced **discriminative classifiers such as CRFs**, and increasingly powerful hybrid and global modelling approaches, the field continues to evolve, leveraging sophisticated features and algorithms to enhance accuracy and efficiency.

Features

Although most approaches are tightly related to the kinds of features employed, it is beneficial for demonstrative purposes to decouple these. Similarly, although most feature categories, such as lexical or prosodic features, are common in sentence and topic segmentation, their usage is very different.